CS 498 DMC — Prototype Presentation

MongoDB + Twitter Dataset Analytics

Team: jtrej7, pa19, runchen3

Date: 2026-04-29 (initial submission draft)

Agenda (10 min + buffer)

1. Problem + prototype scope
2. Data model (with concrete tweet example)
3. Data loading + preprocessing workflow
4. Index strategy and why it matters
5. Live demo plan (at least 2 queries)
6. Critique: what worked / what didn’t
7. What changed from initial design
8. Lessons learned + practical advice
9. Backup slides (extra demos, tradeoffs, risks)

Project Goal and Prototype Scope

- Build a browser-based analytics prototype for Twitter-like data
- Storage system: **MongoDB document model**
- UI stack in repo:
  - TanStack Start + React
  - Netlify deployment target
  - `/queries` page supports **demo mode** interactions for Q1–Q6
- Prototype objective:
  - Validate model + query patterns
  - Demonstrate assignment queries clearly
  - Keep implementation explainable within a short presentation

Why MongoDB for This Dataset

- Source data is nested JSON tweets
- MongoDB stores nested objects naturally (minimal reshape)
- Query-heavy tasks use:
  - Aggregation (`$match` , `$group` , `$sort` , `$project`)
  - Path traversal (`$graphLookup` style logic)
- Main design decision:
  - Keep a **single `tweets` collection** as core fact table
  - Avoid over-normalization and expensive join-like pipelines

Data Model Summary (Core Fields)

Each tweet document keeps:

- Identity / time: `id`, `created_at`, `timestamp_ms`
- Content: `text`, `lang`
- User sub-document: `user.id`, `user.name`, `user.screen_name`, `user.verified`
- Location: `place.country`, `place.full_name`, `place.place_type`
- Entities: `entities.hashtags[]`, `entities.urls[]`, `entities.user_mentions[]`
- Relationship fields: `in_reply_to_status_id`, `in_reply_to_user_id`, `quoted_status_id`, `retweeted_status`
- Engagement counters: `reply_count`, `retweet_count`, `favorite_count`

Example Document (Simplified)

```
{
  "id": 103,
  "created_at": "2026-04-01T10:10:00.000Z",
  "text": "@bob your aggregation idea is neat #nosql #mongodb",
  "user": {
    "id": 1,
    "name": "Black Lucifer",
    "screen_name": "blcklcfr",
    "verified": true
  },
  "place": { "country": "USA" },
  "entities": {
    "hashtags": [{ "text": "nosql" }, { "text": "mongodb" }]
  },
  "in_reply_to_status_id": 106,
  "in_reply_to_user_id": 3,
  "is_quote_status": false
}
```

Validation + Type Consistency

Schema validator enforces important fields/types:

- Required: `id`, `created_at`, `text`, `user`, `entities`
- IDs standardized to numeric long-like type
- `created_at` converted to BSON Date
- `entities.hashtags` forced to array shape
- Missing location represented with explicit `null`

Why this matters:

- Fewer runtime edge cases in aggregation
- Better query predictability
- Cleaner explanation in presentation/demo

Data Loading Pipeline

1. Read line-delimited JSON tweet records
2. Drop malformed records missing essential fields
3. Normalize IDs (`id` , `user.id` , reply/quote IDs)
4. Parse date fields to BSON Date
5. Preserve nested subdocuments (`user` , `entities` , `place`)
6. Bulk insert in batches (`bulk_write` style)
7. Build indexes after insert
8. Run sanity checks (counts, null rates, sample inspection)

Indexing Strategy

```
db.tweets.createIndex({ id: 1 }, { unique: true })
db.tweets.createIndex({ "user.screen_name": 1, created_at: 1 })
db.tweets.createIndex({ in_reply_to_status_id: 1 })
db.tweets.createIndex({ "user.id": 1 })
db.tweets.createIndex({ "user.verified": 1 })
db.tweets.createIndex({ "place.country": 1 })
db.tweets.createIndex({ "entities.hashtags.text": 1 })
```

- Supports filtering/traversal for all 6 assignment queries
- Reduces full scans during groupings and path expansion

Prototype UI in This Repo (What Exists Today)

- `/queries` has two modes:
  - Normal mode: runs predefined query handlers
  - **Demo mode**: UI-only interactive simulation for Q1–Q6
- Demo mode currently includes:
  - Query picker + parameter inputs
  - Pipeline sketch panel
  - “Run Qx” interaction
  - Result table + simulated execution time
- This is great for presentation reliability when DB connectivity is unstable

Demo Walkthrough Plan (at least 2 queries)

Query 1 — Thread reconstruction from `blcklcfr`

- Show graph-style parent traversal via reply IDs
- Output sorted by posting time
- Explain why keeping replies in same collection helps

Query 2 — Country with most tweets

- Group on `place.country`
- Sort desc + limit 1
- Show simple aggregation path and index relevance

Optional Extra Demo (if time allows)

Query 4 — Top hashtags (dedup per tweet)

- Unwind hashtags
- Dedup `(tweet_id, hashtag)` first
- Regroup by hashtag to avoid duplicate inflation

Query 6 — Nature of engagement for verified users

- Classify each tweet as simple/reply/retweet/quote
- Compute percentages per user:
  - $simple\% = \frac{simple}{total} \times 100$
  - $reply\% = \frac{reply}{total} \times 100$
  - etc.

Query Execution Discussion (Critique)

What worked well:

- Aggregation model matched assignment query style
- Single-collection strategy simplified explanations
- Demo mode made interactions deterministic in live presentation

What still hurts:

- Q5 (mutual-reply trios) is computationally heavy
- Graph-style traversals can explode on high-degree nodes
- Hashtag processing still expensive on large cardinality distributions
- Simulated demo != true production latency profile

What We Would Change (If Redesigning)

1. Add a derived/summary layer:
    - materialized country counts
    - precomputed hashtag rollups
2. Introduce ingestion-time denormalized helper fields:
    - lowercased hashtag list
    - reply edge tuples
3. Add index/plan observability from day 1:
    - save explain plans
    - track stage cardinalities
4. Separate “demo reliability mode” and “benchmark mode” explicitly in UI

What Changed from Initial Design

Initial idea: split into multiple collections (`users` , `replies` , etc.)

Observed issue:

- Frequent join-like operations (`$lookup`) across collections
- Higher complexity for recursive thread reconstruction
- Harder to present/maintain in short assignment timeline

Final decision:

- Keep tweets-centric single collection
- Use nested metadata directly
- Leverage MongoDB aggregation + traversal strengths

Lessons Learned (System + Cloud Data Management)

- Data shape alignment beats premature normalization in document DBs
- Type consistency at ingest time saves huge query/debug cost later
- “Working query” and “presentable query” are different deliverables
- Cloud deployment is easy; cloud reliability for demos needs guardrails
- Local demo mode is invaluable when remote DB/network is flaky

Advice for Future Teams

1. Start with two things early:
    - strict preprocessing contract
    - indexing hypotheses per target query
2. Build a tiny but realistic synthetic dataset for demos/tests
3. Add a fallback interaction mode (mock/demo) before final week
4. Keep one slide per query family:
    - objective
    - pipeline sketch
    - bottleneck + mitigation
5. Capture explain-plan screenshots early; don’t wait until final day

Risks, Limitations, and Honest Scope

- Current repo’s demo mode is intentionally UI-simulated
- Real-world scale may require:
  - sharding strategy
  - incremental materialization
  - workload-specific index tuning
- Q5 remains the highest risk for large datasets
- Presentation demo prioritizes clarity and robustness over raw performance benchmarking

Conclusion

- Prototype demonstrates the full analytics story end-to-end:
  - data model
  - ingestion
  - indexed querying
  - interactive query UI
- We can confidently demo at least 2 queries in 10 minutes
- Design is practical for course scope and extensible for future hardening

Thank you — questions welcome

Backup Slide A — Suggested Live Script (Minute-by-Minute)

- 0:00–1:00: problem + stack
- 1:00–2:30: model + schema example
- 2:30–3:30: loading + indexing
- 3:30–6:30: live demo Q1 + Q2
- 6:30–8:00: critique + design changes
- 8:00–9:30: lessons + advice
- 9:30–10:00: summary

Backup Slide B — Query-to-Index Mapping

- Q1 thread traversal → `user.screen_name`, `in_reply_to_status_id`, `id`
- Q2 country max → `place.country`
- Q3 most active user → `user.id`
- Q4 hashtags top-k → `entities.hashtags.text`
- Q5 mutual trios → `user.id`, `in_reply_to_user_id` (derived edge structure helps)
- Q6 engagement split → `user.verified`, plus reply/retweet/quote markers

Backup Slide C — Presenter Notes (Trim Candidates)

Slides that can be cut first if time is tight:

- Optional extra demo slide (Q4/Q6)
- Risks & limitations slide
- Backup A/B/C

Slides that should stay:

- agenda
- model summary + example
- indexing
- demo plan with Q1/Q2
- critique + changed design
- lessons learned

Backup Slide D — Placeholder for Screenshots

Add screenshots before submission if available:

1. Imported data sample in MongoDB Compass/shell
2. `/queries` demo mode with Q1 selected
3. Result table after running Q2
4. (Optional) pipeline sketch panel

Backup Slide E — Submission Checklist

- [ ] Export PDF and verify page breaks
- [ ] Keep font size readable in print
- [ ] Rehearse with 10-minute timer
- [ ] Mark 2 slides as “skip if short on time”
- [ ] Upload initial PDF by 4/29